

# THE METER

Usage-Based Billing Engine for AI APIs

---

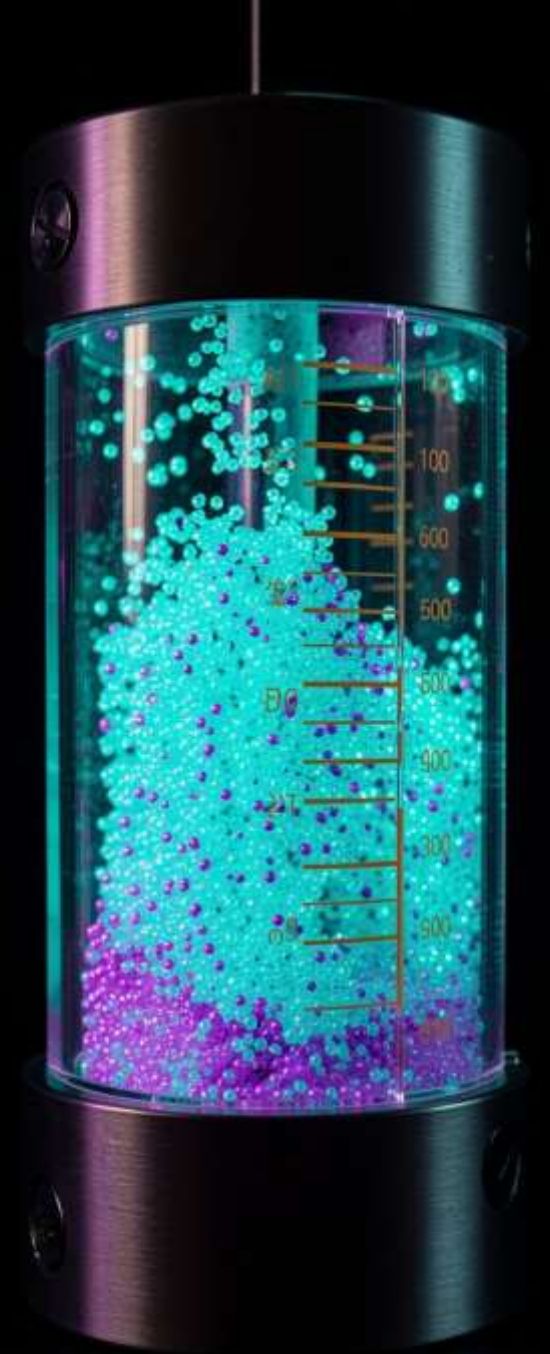
## The Challenge

- AI APIs are billed based on token consumption, making usage tracking critical.
- Build a billing engine for an Indian AI startup, with pricing in ₹ per 1,000 tokens.
- Support volume-based tiered pricing for different usage levels.

- Handle subscription plan changes during an active billing cycle.
- Guarantee that every usage event is processed accurately and exactly once.

## Core Objective

Turn raw API usage into accurate, auditable invoices.



# REVERSE ENGINEERING

## Understanding the Original Flexprice Engine

---

### Our Approach

- Cloned and analyzed the Flexprice open-source billing system.
- Used AI-assisted code exploration to trace the actual implementation.
- Followed the complete path from usage ingestion → metering → pricing → invoicing.
- Investigated how the original handles:

- Event deduplication
- Meter aggregation
- Tiered pricing
- Subscription changes
- Billing periods
- Proration

## Evidence-Driven Analysis

- Every important observation backed by source file + line reference.
- Focused specifically on the three areas required by the Killer Tests.

We didn't guess the behavior — we traced it from the source.

# FROM CODE TO SPECIFICATION

## Building From Documents, Not Assumptions

We converted our reverse-engineering findings into 7 implementation-ready documents:

- OBSERVATIONS → What the original actually does
- PRD → What our system must achieve
- ARCHITECTURE → Components & system flow
- DATA MODEL → Events, meters, plans & invoices
- API → Usage and billing interfaces
- GAPS → Weaknesses + opportunities for improvement
- AGENT LOG → AI-assisted development trail

## Core Architecture

1. Usage Event
2. Idempotency Validation
3. Meter Aggregation
4. Tiered Pricing
5. Plan Proration
6. Invoice Generation

The documentation becomes the single source of truth for the rebuild.

# THE KILLER TESTS

## Three Tests That Define Correctness

---

### 01 — Duplicate Protection

Same usage event arrives multiple times.

Expected:

1. Event → 1 Charge
2. No double billing.

### 02 — Mid-Month Plan Upgrade

Customer switches from a lower plan to a higher plan during the billing cycle.

Expected:

Charges are split according to the actual time spent on each plan.

### 03 — Tiered Pricing

Usage crosses multiple pricing thresholds.

Expected:

Total Bill =  $\sum$  (Usage within tier × Tier Rate)

Engine output must match the manual calculation exactly.

# OUR REBUILD

## Reliable, Testable & Improved

---

### What We Built

- Usage-based billing engine designed specifically for AI token consumption.
- Idempotent event processing prevents duplicate charges.
- Tier-aware pricing engine calculates progressive usage costs.
- Proration engine handles mid-cycle plan changes.
- Configurable time limits through .env for fast automated testing.

### Beyond the Original

Two improvements identified through our GAPS analysis:

**Guaranteed Event Delivery with Synchronous Ingestion  
Error Surfacing & Backpressure  
Complete Default-Deny RBAC & Idempotent  
Subscription Lifecycle**

### Final Outcome

Reverse Engineer → Document → Rebuild → Test → Improve